

HOW MUCH SYNTHETIC LOGIC DOES AN INSTRUCTION MIXTURE NEED?

Anonymous authors

Paper under double-blind review

ABSTRACT

Synthetic training data is almost always written by a language model, which bounds it by what that model knows and passes on its errors. We study a source with no model in it. A program samples a multi-hop deduction problem, solves it, and writes the solution out, so every document is correct by construction. We replace part of an ordinary instruction mixture with such derivations and vary the share from one to twenty-five percent, holding the number of training examples fixed at one hundred thousand and using three random seeds for every configuration. Two effects appear at different doses. One percent of the mixture, one thousand generated documents, takes Qwen2.5-7B from writing no derivation that a checker accepts to writing derivations of fifty-one lines that a checker accepts in more than nine cases out of ten, and this ability extends to chains about twice as long as any seen in training. Accuracy on held-out deduction benchmarks needs a larger dose and then stops improving. At five percent, accuracy on ProofWriter at inference depth three rises from 39.3 to 50.0 percent, and at ten percent of English-rendered derivations it reaches 53.2 percent, while accuracy on FOLIO rises from 51.4 to 57.8 percent. Eight general benchmarks move by less than the spread between seeds. We then explain the gain. Instruction tuning leaves the model answering affirmatively without regard to negation, and the synthetic derivations repair this. The entire improvement lies in claims that are negated and false, where accuracy rises from 17.6 to 56.9 percent, while the three remaining cells of the polarity by label square do not move, and the probability that the model answers “true” falls from 0.73 to 0.50 as the dose grows.

1 INTRODUCTION

The public supply of human-written text is finite, and current projections place its exhaustion for frontier training runs within this decade (Villalobos et al., 2024). The standard remedy is synthetic text, and in nearly every published recipe a language model writes it. Textbook-style corpora (Gunasekar et al., 2023; Eldan & Li, 2023), rephrased web documents (Maini et al., 2024; 2025; Su et al., 2025) and prompted expansions of a small seed corpus (Yang et al., 2025) all take this form. Data of this kind inherits the generating model’s errors and blind spots, and repeated training on it narrows the distribution (Shumailov et al., 2024; Dohmatob et al., 2025). Verified synthetic data avoids the problem in principle (Feng et al., 2025), and individual systems have trained on program-generated and mechanically checked derivations at scale (Trinh et al., 2024).

A few hundred lines of Python suffice to sample a multi-hop deduction problem, solve it, and write the solution out. The resulting documents are correct by construction, unlimited in quantity and free of any teacher model. They are also narrow, repetitive and unlike the web, and they describe a world of abstract constants that carry colour and direction predicates and nothing else. We ask how much of such material an ordinary instruction mixture needs before the model that results reasons measurably better, and we ask what the improvement consists of.

We answer the first question with a dose sweep. We fine-tune Qwen2.5-7B (Yang et al., 2024) on one hundred thousand instruction examples and replace one, five, ten or twenty-five percent of them with program-generated derivations, so that every condition sees the same number of examples and the conditions differ only in the content of the replaced slice. We repeat every configuration with three

random seeds and we render the derivations in two notations, a formal one and a controlled English one, drawn from the same latent proofs.

Two effects appear, and they appear at different doses. At one percent, one thousand documents, the model acquires the ability to write a derivation that a checker accepts. It writes proofs of fifty-one lines at chain length twenty-five, the checker accepts more than nine in ten of them, and the ability extends to chains of length forty-five, about twice the longest chain in training. A model trained on the same mixture without the replacement writes no such derivation in six hundred attempts. At five percent the model also becomes more accurate on held-out deduction benchmarks, and accuracy on ProofWriter (Tafjord et al., 2021) at inference depth three rises from 39.3 to 50.0 percent. Ten percent of English-rendered derivations reaches 53.2 percent and accuracy on FOLIO (Han et al., 2024) rises by 6.4 points. The curve is flat from five percent upward, so a quarter of the mixture buys nothing that a twentieth does not already provide. Across eight general benchmarks, among them GSM8K, MMLU and ARC, the arms differ by less than the spread between seeds of the unmodified control.

We answer the second question by locating the improvement. The instruction mixture alone leaves the model with an affirmative answer prior. It answers that a claim holds in three cases out of four, and it answers a claim and its negation identically in more than a third of items. The synthetic derivations remove this. Splitting the held-out problems by the polarity of the claim and by the gold label, the entire gain lies in one of the four cells, the claims that are negated and false, where accuracy rises from 17.6 to 56.9 percent as the dose grows. The other three cells move by a few points at most, and the probability of answering “true” falls monotonically with dose from 0.73 to 0.50. The improvement is therefore a correction of a decision rule that instruction tuning installs, and we support this reading with a length-matched control, a contamination audit, prior-invariant measures of association and paired significance tests.

Our contributions are the following.

- We measure how much program-generated deduction data an instruction mixture needs. One percent installs the ability to write checkable derivations, five percent is enough for the full benchmark gain, and larger shares add nothing.
- We show that the gain is a repair of an affirmative answer prior left by instruction tuning, and we localise it to a single cell of a polarity by label split of the evaluation set.
- We show that the derivations the model writes are machine-checkable, and we report both where a checker converts sampling into accuracy without an answer key and where it fails to do so.

2 RELATED WORK

Synthetic data written by models. Most synthetic pretraining corpora are produced by prompting a language model, either to write documents in a chosen style (Gunasekar et al., 2023; Eldan & Li, 2023) or to rewrite existing web text (Maini et al., 2024; 2025; Su et al., 2025). The quality of the result is bounded by the generating model, and training repeatedly on such text degrades the distribution (Shumailov et al., 2024; Dohmatob et al., 2025). Our source contains no model, so the ceiling of the generator is the ceiling of the program that writes it.

Synthetic deduction corpora. Several corpora present deduction problems to language models for training or evaluation, among them RuleTaker and ProofWriter (Clark et al., 2020; Tafjord et al., 2021), PrOntoQA (Saparov & He, 2023) and FLD (Morishita et al., 2023; 2024). These corpora render their proofs in English. Zhou et al. (2025) compare symbolic and English supervision on matched problems and report that English supervision generalises further. We generate both renderings from one latent proof, which lets us hold the problem and the answer fixed while varying only the notation, and we find that the ordering depends on the depth of the training proofs.

Verifiable rewards and checkable traces. Training against a verifier has become a standard recipe for mathematics and code (Lambert et al., 2025; Yue et al., 2025), and formal systems allow a proof to be checked mechanically (Trinh et al., 2024). Our derivations are checkable by a small

forward-chaining program, which lets us ask whether a checker can select a correct sample without an answer key, and lets us measure how far that selection carries to a human-written benchmark.

Answer priors in tuned models. Instruction-tuned models are known to acquire response biases that are separable from their underlying competence (Ouyang et al., 2022), and negation in particular remains difficult for language models (Kassner & Schütze, 2020; Hosseini et al., 2021; Truong et al., 2023). Our mechanistic result connects these two literatures. The pathology that synthetic deduction data repairs is an affirmative answer prior that instruction tuning installs, and the repair is visible as a change in the prior and not as a uniform improvement in accuracy.

3 DERIVATIONS FROM A PROGRAM

The generator samples a set of constants, a set of unary predicates and a set of implication rules, chooses a target chain length, and constructs a problem whose answer follows from the premises by that many applications of modus ponens. It then solves the problem by forward chaining and writes the solution out. Nothing in the pipeline involves a language model, and every document is correct by construction. The vocabulary is deliberately impoverished. Constants are named c_0 through c_{25} and the predicates are colour and direction words, so the documents carry no world knowledge and share almost no vocabulary with the benchmarks we later evaluate.

Each latent proof can be written in two ways. The formal rendering declares the constants and the predicate symbols, states the premises as implications between atoms, and lists the proof as numbered lines with the rule that justifies each one. The English rendering states the same premises and the same proof as ordinary sentences. The two renderings share the prompt, the answer and the proof graph, so a comparison between them isolates notation.

We build every training mixture by replacement. A condition with a share p contains $(1 - p) \cdot 10^5$ examples of the instruction corpus and $p \cdot 10^5$ rendered derivations, so all conditions train on exactly one hundred thousand examples and differ only in the content of the replaced slice. The evaluation split of every mixture is drawn from the instruction corpus alone. This design removes the most common confound in data-augmentation experiments, which is that the treated condition simply sees more data.

We audit the overlap between the generated documents and the evaluation sets. No evaluation item shares an eight-gram with any training derivation, and the overlap of the two hundred most frequent words is four words. Words that the benchmarks rely on, among them “not”, “someone” and the animal nouns of ProofWriter, never occur in the training derivations.

4 HOW MUCH SYNTHETIC LOGIC AN INSTRUCTION MIXTURE NEEDS

4.1 SETUP

We fine-tune Qwen2.5-7B on each mixture for one epoch with a learning rate of 5×10^{-6} , a global batch of one hundred and twenty-eight sequences and a context window of eight thousand one hundred and ninety-two tokens. The window is set from measurement. The longest rendered derivation in the twenty-five hop band occupies seven thousand seven hundred tokens, and truncation is fail-closed, so a run aborts if more than half a percent of its examples would be cut. We train three seeds for every configuration and report all of them.

We evaluate on ProofWriter under the open-world assumption at inference depths zero through five, on FOLIO, on the deduction problems of the generator at chain lengths five through twenty-five, and on eight general benchmarks. For the generator’s own problems we prompt the model in the document format it was trained on, since a derivation has to be requested before it can be checked.

4.2 RESULTS

Table 1 reports the sweep. Two effects separate cleanly.

The first effect is the ability to write a derivation that a checker accepts, and it is complete at one percent of the mixture. A model trained without derivations writes none in six hundred attempts. A

Table 1: Dose sweep on Qwen2.5-7B. Every condition trains on one hundred thousand instruction examples and differs only in the content of the replaced share. We report all three seeds for ProofWriter at inference depth three and the mean elsewhere. The column headed “derivation” gives the fraction of the generator’s own problems at chain length twenty-five for which a checker accepts the written proof and the answer is correct.

Notation	Share	ProofWriter d3 (3 seeds)	Mean	FOLIO	Derivation
none	0%	.388 / .390 / .402	.393	.514	.003
formal	1%	.412 / .392 / .410	.405	.530	.932
formal	5%	.500 / .494 / .506	.500	.578	.993
formal	10%	.490 / .482 / .486	.486	.574	.995
formal	25%	.516	.516	.562	1.000
English	1%	.402 / .404 / .402	.403	.540	.983
English	5%	.492 / .488	.490	.560	.995
English	10%	.546 / .528 / .522	.532	.550	1.000

model trained with one thousand of them writes a derivation in every attempt, and a forward-chaining checker accepts more than nine in ten at chain length twenty-five. The proofs are fifty-one lines long at that depth, which is two lines for every hop plus the premises, and the ability extends to chains of length forty-five, about twice the longest chain in training, where the checker still accepts nine in ten.

The second effect is accuracy on held-out benchmarks, and it needs a larger dose. At one percent, accuracy on ProofWriter at depth three rises by about a point, which is within the spread between seeds. At five percent it rises from 39.3 to 50.0 percent, and at ten percent of English-rendered derivations it reaches 53.2 percent. Every seed of every condition at five percent or more scores above every seed of the control, and the spread among the three control seeds is 1.4 points. Accuracy on FOLIO rises from 51.4 to 57.8 percent. Beyond five percent the curve is flat, and a condition with a quarter of the mixture replaced scores 51.6 percent, within noise of the five percent condition.

The cost of the intervention is not measurable in our evaluation. Across GSM8K, BBH, MMLU, ARC-Challenge, LogiQA, HellaSwag, PIQA and WinoGrande, no condition differs from the control by more than the spread between control seeds.

5 WHY THE GAIN OCCURS

5.1 AN AFFIRMATIVE ANSWER PRIOR

ProofWriter presents a theory and a claim and asks whether the claim is true, false or unknown. We split the evaluation items by two properties that the model can read off the prompt, whether the claim is negated and what the gold label is, and we measure accuracy in each of the four resulting cells for decidable items. Table 2 reports the split.

The control answers correctly on 98.5 percent of claims that are negated and true, on 68 percent of claims that are positive and false, and on 75 percent of claims that are positive and true. On claims that are negated and false it answers correctly on 17.6 percent. It is therefore not failing to reason. It is answering that the claim holds, and it does so in three cases out of four across the whole evaluation set, while the gold labels are close to balanced.

The synthetic derivations correct this. As the dose grows, accuracy on negated and false claims rises from 17.6 to 56.9 percent, the probability of answering “true” falls monotonically from 0.73 to 0.50, and the three remaining cells move by a few points at most. The large cell of positive and true claims drifts down by about eight points, which is what a correction of an affirmative prior predicts. The improvement is a change in a decision rule and it is visible as such.

5.2 CONTROLS

Three controls bound what else could produce the effect.

Table 2: Accuracy by claim polarity and gold label on decidable ProofWriter items, pooled over inference depths one through five, formal notation, mean over seeds. The rightmost column is the probability that the model answers “true” on the full evaluation set. The gain lies in one cell.

Share	negated, false $n = 785$	negated, true $n = 67$	positive, false $n = 112$	positive, true $n = 789$	$P(\text{answers true})$
0%	.176	.992	.683	.752	.735
1%	.241	.985	.649	.716	.692
5%	.501	.920	.830	.687	.565
10%	.496	.945	.774	.664	.552
25%	.569	.910	.679	.668	.502

The first is length and domain. In a midtraining study on the same generator we replaced the same share of the corpus with ordinary documents drawn from the pretraining mixture and matched to the derivations in length distribution. That condition is indistinguishable from the control on ProofWriter, with a paired test giving $p = 0.62$ and $p = 1.00$ in two seeds, while the derivation condition gains 12.6 points. Length and domain therefore do not explain the gain.

The second is memorisation. No evaluation item shares an eight-gram with any training derivation, and the vocabulary overlap is four words in the two hundred most frequent.

The third is the role of deductive order. We midtrain a further condition on the same derivations with the lines of each proof shuffled, which preserves every token, the vocabulary, the length and the conclusion and destroys only the order in which facts follow from one another. This condition is in training at the time of writing and we will report it with the same measures.

5.3 WHAT THE GAIN IS NOT

If the improvement were simply that the model reasons more, then asking the control to reason aloud should close the gap. It does not. Under a prompt that requests a chain of reasoning before the answer, the control reaches 39.9 percent on ProofWriter at depth three, the five percent condition reaches 45.3 percent and the ten percent condition reaches 49.4 percent. A condition with a quarter of the mixture replaced reaches 55.8 percent, so the advantage grows with dose in the regime where a well-prompted baseline is the natural objection.

6 CHECKABLE DERIVATIONS

A derivation written in a fixed format can be checked by a program. We implement a forward-chaining checker that reads the premises of an item and accepts a proof line when it is a premise or follows from lines already established. The checker reads both notations, translating formal lines through the predicate legend that the model itself writes. On ProofWriter it reproduces the published gold label on 99, 95, 91, 91 and 84 percent of items at inference depths zero through five, and its residual errors are answers of “unknown” where the gold label is decided, so it under-credits.

On the generator’s own problems the checker is close to an oracle. A derivation that the checker accepts carries the correct answer in 99 percent of samples, and selecting the first accepted derivation out of sixteen samples gives the same accuracy as asking whether any of the sixteen is correct. A gold-free selection rule therefore matches sixteen-sample coverage.

On ProofWriter the same rule is much weaker. Sampling sixteen derivations per item at temperature 0.8, between 28 and 37 percent carry a proof that the checker accepts, against none for the control, and the accuracy of an accepted derivation is between 53 and 69 percent. Selection by the checker therefore trails majority voting over the same samples. The reason is structural. The checker certifies that the written steps follow from the premises, and ProofWriter asks in a third of its items whether a claim is underivable, which a forward chain never demonstrates. A checker pays where it is complete with respect to the task and not merely sound.

7 WHEN NOTATION MATTERS

The dose sweep shows the two notations transferring about equally, with the English rendering ahead at ten percent and the formal rendering ahead at five. A controlled study on the generator alone shows that the ordering is not fixed and that the depth of the training proofs decides it. Fine-tuning OLMo-3-7B on fifty thousand paired examples per condition, English supervision generalises further when the training proofs reach at most twenty hops. At twenty-five hops the ordering reverses, and formal supervision reaches 75.0 ± 9.5 percent out-of-distribution accuracy against 17.8 ± 4.7 for English. Sixteen samples move the reversal to fifteen hops, so formal supervision first widens the set of problems the model can reach and only later improves its first attempt.

The two settings agree once the regime is stated. The formal advantage belongs to dense and deep supervision, and an instruction mixture with a tenth of its examples replaced is neither. We therefore do not recommend the formal rendering on the strength of the controlled study, and we report the two notations as interchangeable at the doses that a practitioner would use.

8 TRAINING STAGE

The same derivations can enter during midtraining instead. We midtrained Qwen2.5-7B on 2.5 billion tokens with a tenth of the mixture replaced, then applied the same instruction tuning to every condition. That route also produces the ProofWriter gain, 12.6 points for the English rendering at inference depth three with two instruction-tuning seeds, and it also leaves the general benchmarks flat, with the exception of a loss of two to three points on LogiQA that replicates across seeds.

Comparing the two routes, midtraining is the more expensive and buys no more. The instruction-tuning route reaches the same accuracy, generalises further on long chains, and does not carry the LogiQA cost. Applying both in sequence adds nothing to either, and on the generator’s own problems the model trained in both stages performs as the midtrained model alone. What the midtrained model retains is an ability that instruction tuning without derivations removes completely. Prompted in the format of its training documents, the midtrained model writes checkable proofs to chain length twenty-five, and after ordinary instruction tuning it writes none.

9 LIMITATIONS

Our gains are confined to deduction. ProofWriter and FOLIO improve, the eight general benchmarks do not move, and we have no evidence that the intervention helps tasks that do not ask for a chain of inferences.

The mechanism we identify is a correction of an answer prior. This is a real and replicated effect, and it is a narrower claim than an improvement in reasoning capability. A model whose prior is already balanced may gain less.

Our checker is sound and incomplete. It certifies that written steps follow and it cannot certify that a fact is underivable, which limits the benefit of checker-based selection on benchmarks with an open-world answer.

We train one model family at one scale. The affirmative prior that the derivations repair is a property of the instruction corpus we use, and a different instruction corpus may leave a different pathology.

10 CONCLUSION

A program that writes deduction problems and their solutions produces training data with no teacher model in it, and a small amount of that data changes what an instruction-tuned language model does. One percent of an ordinary mixture is enough for the model to write proofs that a checker accepts, at chain lengths about twice those it saw in training, where a model trained without the replacement writes none. Five percent is enough for the full improvement on held-out deduction benchmarks, and larger shares add nothing. The improvement is a correction of an affirmative answer prior that instruction tuning installs, it is confined to the claims where that prior does damage, and it costs nothing on the general benchmarks we measure.

REFERENCES

- Peter Clark, Oyvind Tafjord, and Kyle Richardson. Transformers as soft reasoners over language. In *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence*, pp. 3882–3890, 2020. doi: 10.24963/ijcai.2020/537.
- Elvis Dohmatob, Yunzhen Feng, Arjun Subramonian, and Julia Kempe. Strong model collapse. In *Proceedings of the Thirteenth International Conference on Learning Representations (ICLR 2025)*, 2025. doi: 10.48550/arXiv.2410.04840.
- Ronen Eldan and Yuanzhi Li. TinyStories: How small can language models be and still speak coherent English? *arXiv preprint arXiv:2305.07759*, 2023. doi: 10.48550/arXiv.2305.07759.
- Yunzhen Feng, Elvis Dohmatob, Pu Yang, François Charton, and Julia Kempe. Beyond model collapse: Scaling up with synthesized data requires verification. In *Proceedings of the Thirteenth International Conference on Learning Representations (ICLR 2025)*, 2025. doi: 10.48550/arXiv.2406.07515.
- Suriya Gunasekar, Yi Zhang, Jyoti Aneja, Caio César Teodoro Mendes, Allie Del Giorno, Sivakanth Gopi, Mojan Javaheripi, Piero Kauffmann, Gustavo de Rosa, Olli Saarikivi, et al. Textbooks are all you need. *arXiv preprint arXiv:2306.11644*, 2023. doi: 10.48550/arXiv.2306.11644.
- Simeng Han, Hailey Schoelkopf, Yilun Zhao, Zhenting Qi, Martin Riddell, Wenfei Zhou, James Coady, David Peng, Yujie Qiao, Luke Benson, Lucy Sun, Alex Wardle-Solano, Hannah Szabo, Ekaterina Zubova, Matthew Burtell, Jonathan Fan, Yixin Liu, Brian Wong, Malcolm Sailor, Ansong Ni, Linyong Nan, Jungo Kasai, Tao Yu, Rui Zhang, Alexander R. Fabbri, Wojciech Kryscinski, Semih Yavuz, Ye Liu, Xi Victoria Lin, Shafiq Joty, Yingbo Zhou, Caiming Xiong, Rex Ying, Arman Cohan, and Dragomir Radev. FOLIO: Natural language reasoning with first-order logic. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 22017–22031, 2024. doi: 10.18653/v1/2024.emnlp-main.1229.
- Arian Hosseini, Siva Reddy, Dzmitry Bahdanau, R. Devon Hjelm, Alessandro Sordoni, and Aaron Courville. Understanding by understanding not: Modeling negation in language models. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics*, pp. 1301–1312, 2021. doi: 10.18653/v1/2021.naacl-main.102.
- Nora Kassner and Hinrich Schütze. Negated and misprimed probes for pretrained language models: Birds can talk, but cannot fly. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pp. 7811–7818, 2020. doi: 10.18653/v1/2020.acl-main.698.
- Nathan Lambert, Jacob Morrison, Valentina Pyatkin, Shengyi Huang, Hamish Ivison, Faeze Brahman, Lester James V. Miranda, Alisa Liu, Nouha Dziri, Shane Lyu, Yuling Gu, Saumya Malik, Victoria Graf, Jena D. Hwang, Jiangjiang Yang, Jan A. Botha, Akshita Bhagia, Hailey Schoelkopf, Khyathi Chandu, Benjamin Newman, Luca Soldaini, Luke Zettlemoyer, Noah A. Smith, Yejin Choi, Ali Farhadi, and Hannaneh Hajishirzi. TULU 3: Pushing frontiers in open language model post-training. In *Proceedings of the 2025 Conference on Language Modeling*, 2025.
- Pratyush Maini, Skyler Seto, Richard Bai, David Grangier, Yizhe Zhang, and Navdeep Jaitly. Rephrasing the web: A recipe for compute and data-efficient language modeling. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 14044–14072, 2024. doi: 10.18653/v1/2024.acl-long.757.
- Pratyush Maini, Vineeth Dorna, Parth Doshi, Aldo Carranza, Fan Pan, Jack Urbanek, Paul Burstein, Alex Fang, Alvin Deng, Amro Abbas, et al. BeyondWeb: Lessons from scaling synthetic data for trillion-scale pretraining. *arXiv preprint arXiv:2508.10975*, 2025. doi: 10.48550/arXiv.2508.10975.
- Terufumi Morishita, Gaku Morio, Atsuki Yamaguchi, and Yasuhiro Sogawa. Learning deductive reasoning from synthetic corpus based on formal logic. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pp. 25254–25274. PMLR, 2023. doi: 10.48550/arXiv.2308.07336.
- Terufumi Morishita, Gaku Morio, Atsuki Yamaguchi, and Yasuhiro Sogawa. Enhancing reasoning capabilities of LLMs via principled synthetic logic corpus. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, 2024. doi: 10.48550/arXiv.2411.12498.

- Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul F. Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, volume 35, pp. 27730–27744, 2022.
- Abulhair Saparov and He He. Language models are greedy reasoners: A systematic formal analysis of chain-of-thought. In *International Conference on Learning Representations*, 2023. doi: 10.48550/arXiv.2210.01240.
- Ilia Shumailov, Zakhar Shumaylov, Yiren Zhao, Nicolas Papernot, Ross Anderson, and Yarin Gal. AI models collapse when trained on recursively generated data. *Nature*, 631:755–759, 2024. doi: 10.1038/s41586-024-07566-y.
- Dan Su, Kezhi Kong, Ying Lin, Joseph Jennings, Brandon Norick, Markus Kliegl, Mostofa Patwary, Mohammad Shoeybi, and Bryan Catanzaro. Nemotron-CC: Transforming Common Crawl into a refined long-horizon pretraining dataset. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 2459–2475, 2025. doi: 10.18653/v1/2025.acl-long.123.
- Oyvind Tafjord, Bhavana Dalvi, and Peter Clark. ProofWriter: Generating implications, proofs, and abductive statements over natural language. In *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, pp. 3621–3634, 2021. doi: 10.18653/v1/2021.findings-acl.317.
- Trieu H. Trinh, Yuhuai Wu, Quoc V. Le, He He, and Thang Luong. Solving olympiad geometry without human demonstrations. *Nature*, 625:476–482, 2024. doi: 10.1038/s41586-023-06747-5.
- Thinh Hung Truong, Timothy Baldwin, Karin Verspoor, and Trevor Cohn. Language models are not naysayers: An analysis of language models on negation benchmarks. In *Proceedings of the 12th Joint Conference on Lexical and Computational Semantics*, pp. 101–114, 2023. doi: 10.18653/v1/2023.starsem-1.10.
- Pablo Villalobos, Anson Ho, Jaime Sevilla, Tamay Besiroglu, Lennart Heim, and Marius Hobbhahn. Position: Will we run out of data? limits of LLM scaling based on human-generated data. In *Proceedings of the 41st International Conference on Machine Learning*, pp. 49523–49544, 2024. doi: 10.48550/arXiv.2211.04325.
- An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, et al. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024. doi: 10.48550/arXiv.2412.15115.
- Zitong Yang, Neil Band, Shuangping Li, Emmanuel Candès, and Tatsunori Hashimoto. Synthetic continued pretraining. In *International Conference on Learning Representations*, 2025. doi: 10.48550/arXiv.2409.07431.
- Yang Yue, Zhiqi Chen, Rui Lu, Andrew Zhao, Zhaokai Wang, Yang Yue, Shiji Song, and Gao Huang. Does reinforcement learning really incentivize reasoning capacity in LLMs beyond the base model? In *Advances in Neural Information Processing Systems* 38, 2025. doi: 10.48550/arXiv.2504.13837.
- Yujun Zhou, Jiayi Ye, Zipeng Ling, Yufei Han, Yue Huang, Haomin Zhuang, Zhenwen Liang, Kehan Guo, Taicheng Guo, Xiangqi Wang, et al. Dissecting logical reasoning in LLMs: A fine-grained evaluation and supervision study. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pp. 17075–17098, 2025. doi: 10.18653/v1/2025.findings-emnlp.926.